

Задача A1 отчёт

Ссылка на публичный репозиторий:

<https://github.com/samstarkov/set9-algo/>

ID посылок Codeforces:

1. A1m: [375975578](#)
2. A1q: [375931260](#)
3. A1r: [375951608](#)
4. A1rq: [375952341](#)
5. Обычный MergeSort: [375944867](#)
6. Обычный QuickSort: [375945956](#)

MergeSort.h; QuickSort.h; MergeSortLCP.h; TernaryQuickSort.h; MSDRadixSort.h; HybridRadixSort.h; - файлы содержащие реализации каждого из необходимых алгоритмов, названия соответствуют алгоритмам, содержащимся в файлах. Каждый из файлов содержит по две реализации, одна без подсчёта количества посимвольных сравнений, вторая - с подсчётом.

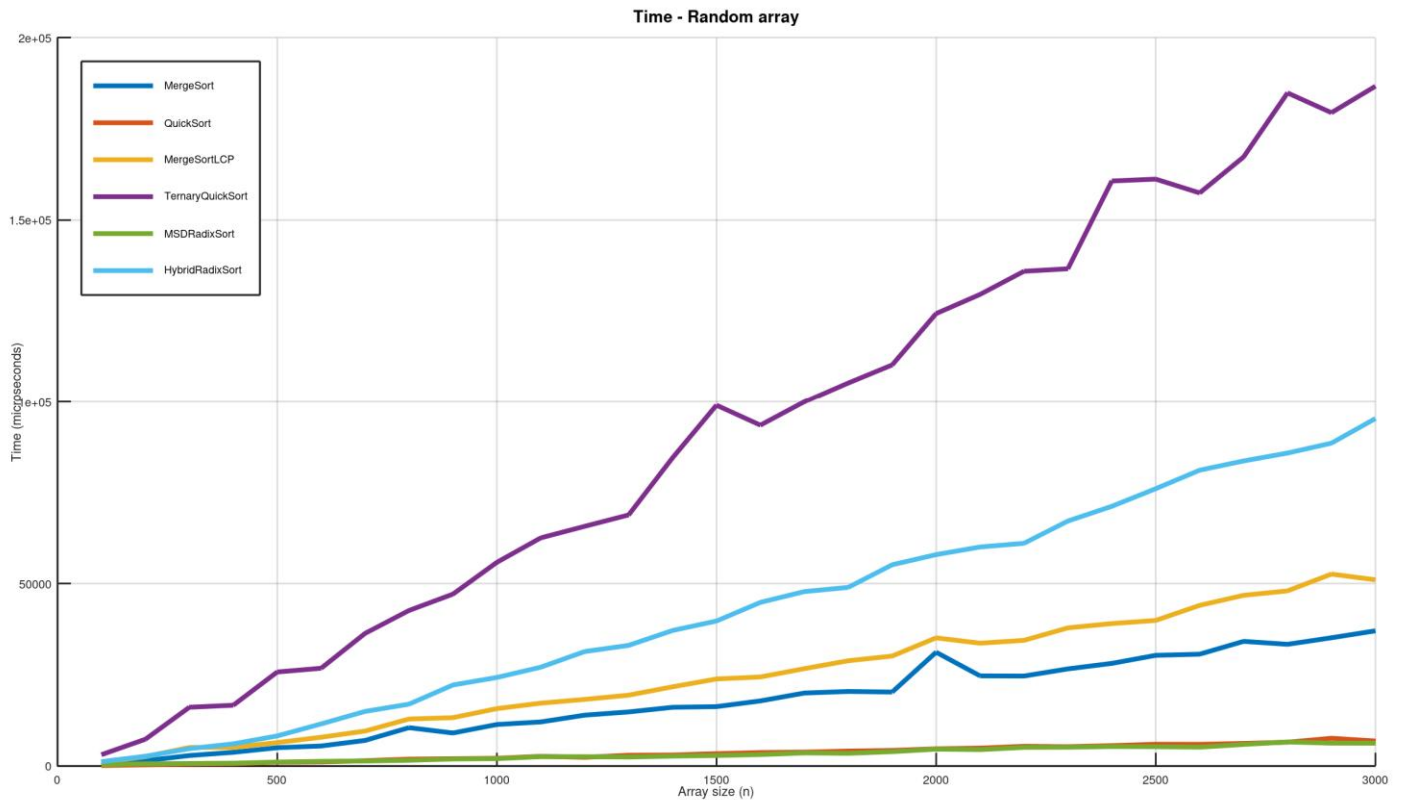
set9_A1.cpp; StringGenerator.h; StringSortTester.h; - файлы содержащие код для получения данных для дальнейшего анализа (составления графиков). Класс **StringGenerator** - класс для генерации массивов случайных строк. Класс **StringSortTester** - класс, содержащий функций измерения производительности тестируемых алгоритмов сортировки строк.

TimeRandom.txt; TimeReversed.txt; TimeAlmostSorted.txt; - файлы с результатами замеров времени для всех шести алгоритмов для трёх видов массивов. Каждый файл содержит семь столбцов, первый из которых - значения n, каждый из следующих - результат замера времени для одного из алгоритмов (в микросекундах). Порядок алгоритмов в файле: MergeSort, QuickSort, MergeSortLCP, TernaryQuickSort, MSDRadixSort, HybridRadixSort.

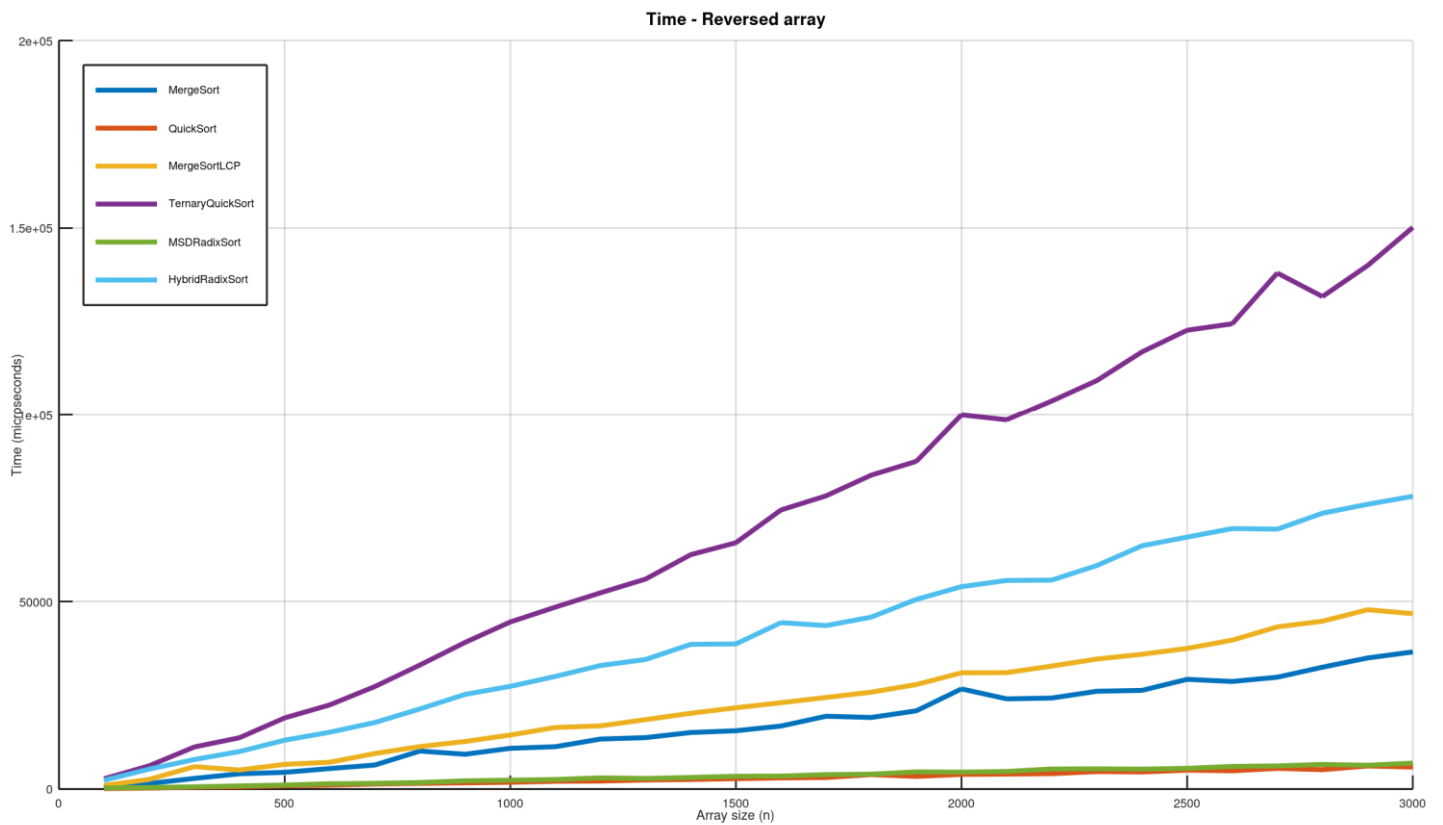
ComparisonsRandom.txt; ComparisonsReversed.txt; ComparisonsAlmostSorted.txt; - файлы с результатами замеров количества посимвольных сравнений для всех шести алгоритмов для трёх видов массивов. Каждый файл содержит семь столбцов, первый из которых - значения n, каждый из следующих - результат замера количества посимвольных сравнений для одного из алгоритмов (в микросекундах). Порядок алгоритмов в файле: MergeSort, QuickSort, MergeSortLCP, TernaryQuickSort, MSDRadixSort, HybridRadixSort.

Далее, в Octave (MatLab) написан код, для представления графиков, необходимых по условию задачи (файл **set9_A1_plots.m**):

Время работы алгоритмов для случайного набора строк:

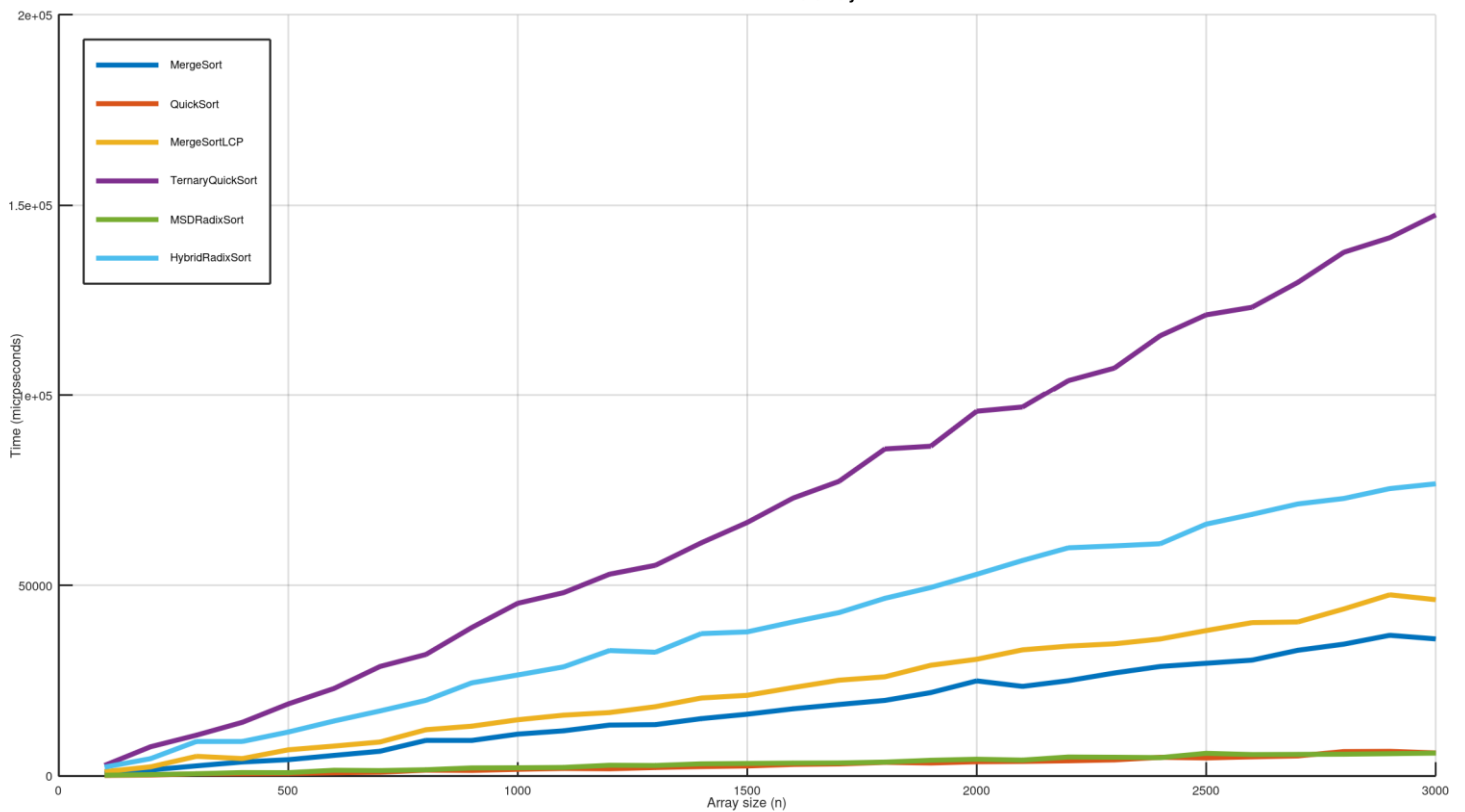


Время работы алгоритмов для обратно отсортированного набора строк:



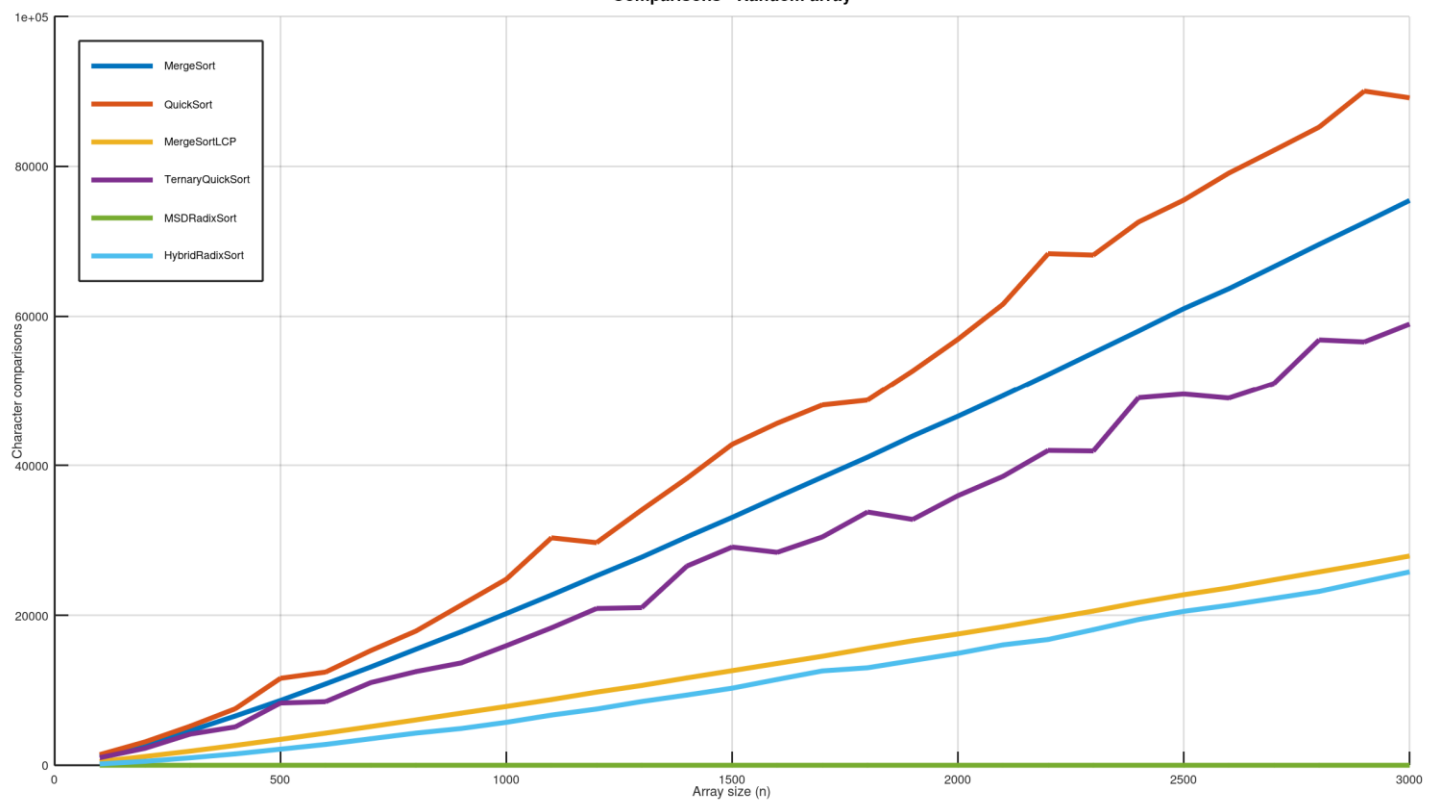
Время работы алгоритмов для почти отсортированного набора строк:

Time - Almost sorted array

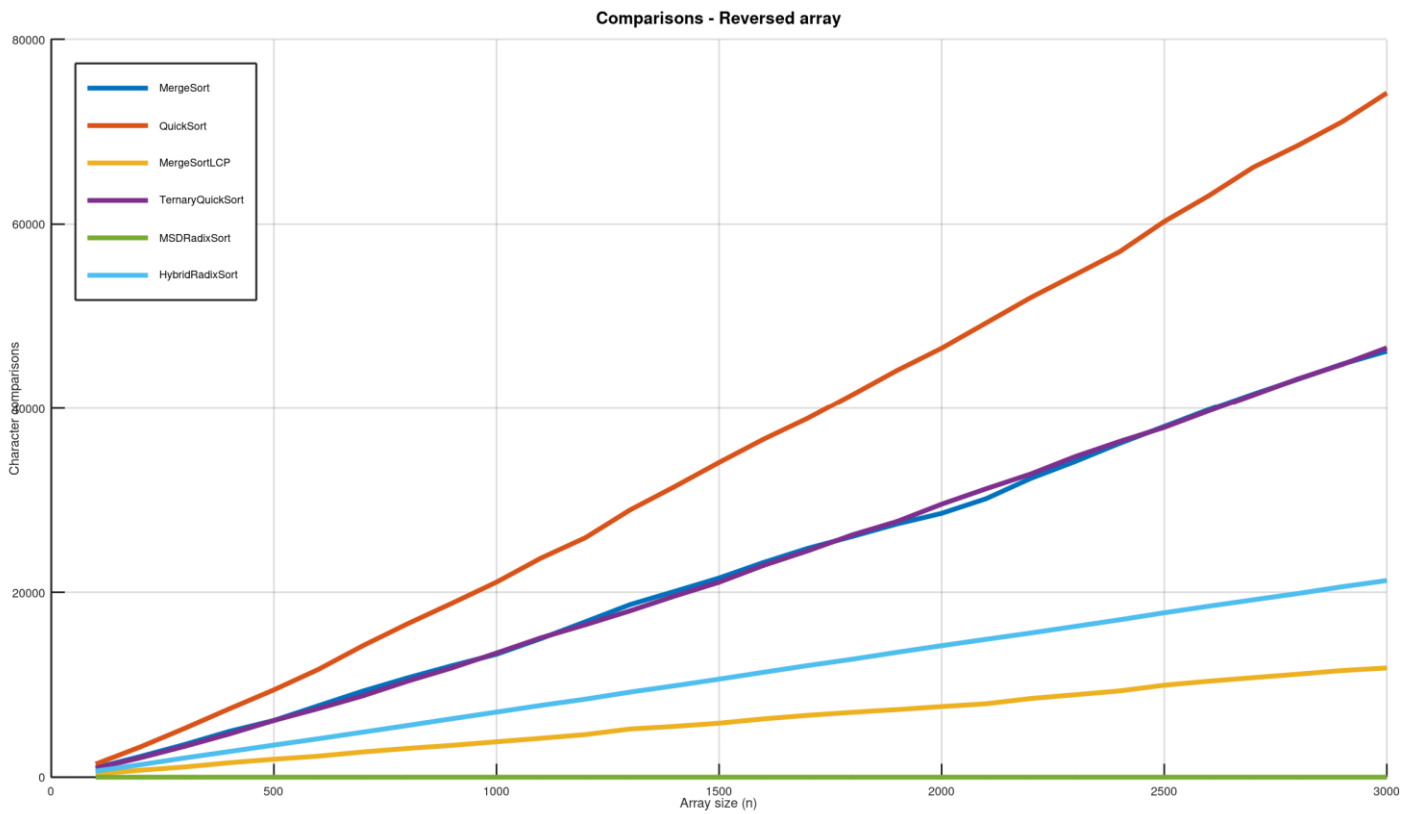


Количество посимвольных сравнений в алгоритмах для случайного набора строк:

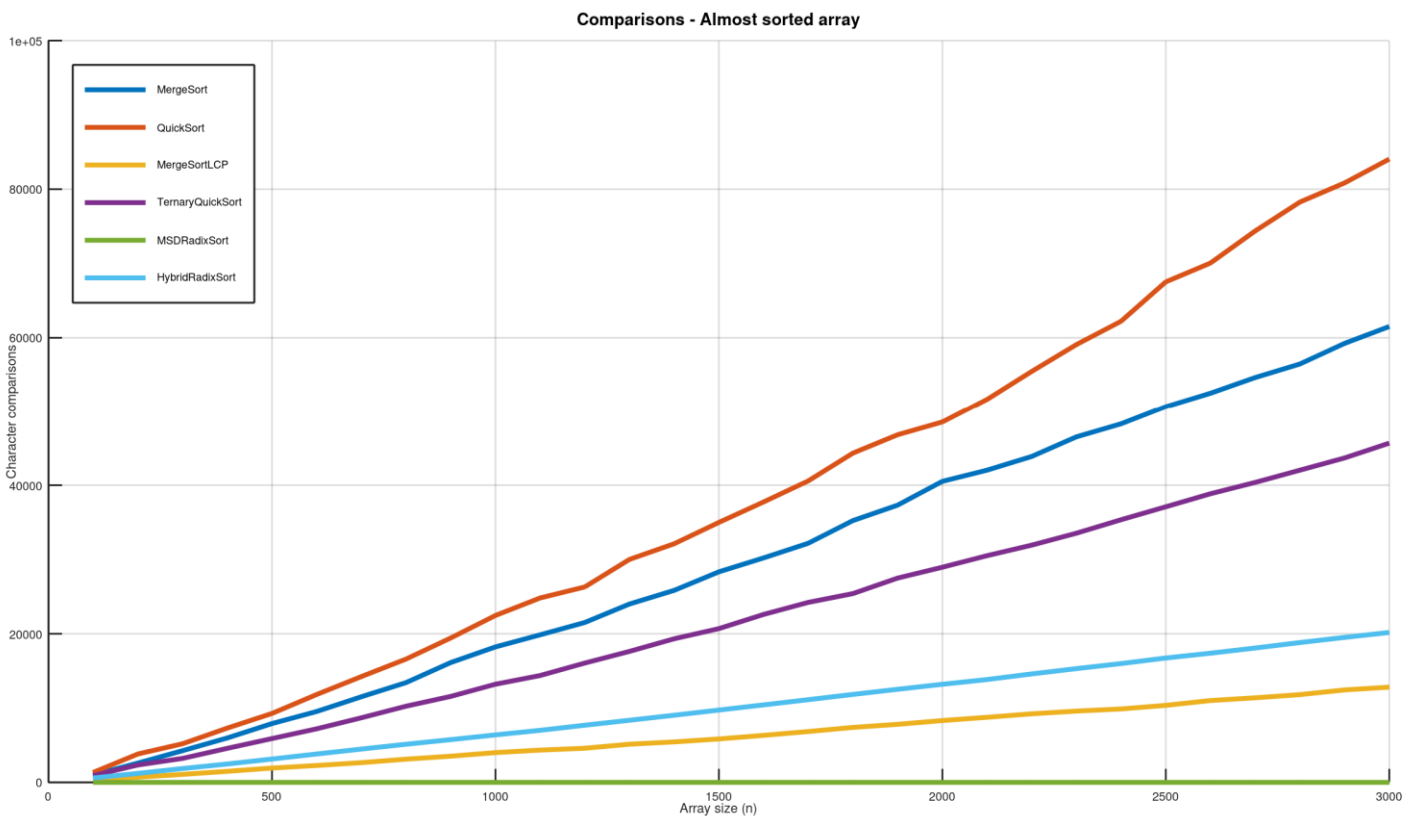
Comparisons - Random array



Количество посимвольных сравнений в алгоритмах для обратно отсортированного набора строк:



Количество посимвольных сравнений в алгоритмах для почти отсортированного набора строк:



Анализ

Время работы

Для всех трёх видов разбираемых наборов строк мы видим примерно идентичные результаты замеров времени, алгоритмы можно расположить в таком порядке по времени работы:

1-2. MSDRadixSort и QuickSort - оба алгоритмы работают достаточно быстро, на графиках разница едва заметна.

3. MergeSort

4. MergeSortLCP

5. HybridRadixSort - хоть MSDRadixSort работает достаточно быстро, но из-за перехода на TernaryQuickSort (по результатам - самый медленный), алгоритм получился не слишком эффективным по времени

6. TernaryQuickSort - самый медленный из алгоритмов

Количество сравнений

Алгоритм MSDRadixSort не делает посимвольных сравнений вообще, поэтому результат их всегда 0.

Для случайных наборов строк вторым по количеству посимвольных сравнений (идём по возрастанию количества) идёт HybridRadixSort, а затем MergeSortLCP. А для обратно отсортированных и почти отсортированных наборов строк меньшее количество сравнений у MergeSortLCP, а большее у HybridRadixSort.

Это вызвано тем, что MSD-фаза в HybridRadixSort работает без сравнений вообще, а TernaryQuickSort подключается только на маленьких подмассивах. Поэтому для случайных наборов строк у HybridRadixSort меньше посимвольных сравнений. А для обратно отсортированных и почти отсортированных MergeSortLCP выигрывает за счёт изначальных больших общих префиксов между соседними строками - сравнения начинаются не с нуля, а с уже известного совпадающего префикса.

Далее для случайных наборов и почти отсортированных наборов строк идёт TernaryQuickSort, а затем MergeSort. Но для обратно отсортированных строк результаты практически идентичны.

Больше всего сравнений у обычного QuickSort т. к. в отличие от MergeSort, где каждые две строки сравниваются ровно один раз при слиянии, в QuickSort одна и та же строка сравнивается с разными опорными элементами на каждом уровне рекурсии, что это даёт больше суммарных сравнений.

Исходя из полученных результатов, можно заметить, что достаточно оптимальным является MSDRadixSort т. к. он работает достаточно быстро и вообще не делает посимвольных сравнений из-за особенности его реализации. Обычный QuickSort, также достаточно быстрый, однако при его выполнении происходит много посимвольных

сравнений. Самым же неоптимальным, исходя из результатов, получился TernaryQuickSort - он получился самым медленным по времени и входит в топ 3 по количеству посимвольных сравнений.